

AWS DEVOPS INTERVIEW GUIDE

EC2 Scenarios (Scenario 1)

Scenario 1: Production Application is Down (Most Asked AWS Interview Question)

Interview Question

"It's 2:00 AM and you receive a P1 alert. The production application is down. Users are unable to access the website. The application is hosted on an EC2 instance behind an ALB. How will you troubleshoot?"

This is probably the **#1 most asked AWS scenario** in senior DevOps interviews.

My first step is **not** logging into the EC2 instance.

First, I gather information.

I ask:

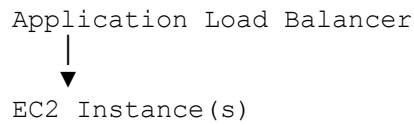
- Is the outage affecting all users or only a region?
- Is the website completely down or just slow?
- When did the issue start?
- Was there any deployment in the last hour?
- Any infrastructure changes (Terraform, CloudFormation)?
- Is only one instance affected or all instances?
- Did CloudWatch generate any alarms?

This helps determine whether it's an application issue, infrastructure issue, or deployment issue.

Step 1 – Check the ALB

Architecture:

Users
↓
▼



The first AWS component I check is the **Application Load Balancer**.

Why?

Because users are connecting to the ALB—not directly to the EC2 instance.

I verify:

- Is the ALB healthy?
- Is the Listener configured correctly?
- Is the Target Group healthy?
- Are all targets unhealthy?

If all targets are unhealthy, the issue is likely on the application or EC2 side.

If only one target is unhealthy, it may be an instance-specific issue.

Check Target Health

AWS Console:

```
EC2
→ Load Balancers
→ Target Groups
→ Targets
```

Possible result:

Instance-1

Status

Unhealthy

Now I inspect the reason.

Example:

Health Check Failed

or

Connection Timeout

Interview Tip

If the interviewer says:

"ALB is healthy but users cannot access the website."

This means the issue is **probably not with the ALB**.

Move to the next layer.

Step 2 – Check EC2 Health

Go to:

EC2
→ Instances

Check:

Status Checks

There are two important checks.

1. System Status Check

AWS verifies:

- Underlying hardware
- Physical server
- AWS network
- Power
- Hypervisor

Example:

System Check Failed

This indicates an AWS infrastructure issue.

Possible actions:

- Stop/Start the instance (if supported).
- Replace the instance.
- Launch a new instance from the AMI.
- Contact AWS Support if required.

2. Instance Status Check

AWS verifies the guest operating system.

Possible causes:

- Kernel panic
- Boot failure
- Corrupt filesystem
- CPU lockup
- Out-of-memory conditions

Example:

```
Instance Status Check Failed
```

This points to an operating system issue rather than AWS hardware.

Step 3 – Review CloudWatch Metrics

Before logging in, I review CloudWatch metrics because they provide historical visibility.

I check:

- CPU Utilization
 - Network In
 - Network Out
 - Disk Read Operations
 - Disk Write Operations
 - Status Check Failed
 - Memory Utilization (if CloudWatch Agent is installed)
 - Disk Usage (if CloudWatch Agent is installed)
-

Example 1

CPU:

```
100%
```

```
for
```

```
30 minutes
```

Possible causes:

- Infinite loop
 - High traffic
 - Java garbage collection
 - Cryptocurrency miner
 - DDoS attack
 - Application bug
-

Example 2

CPU:

0%

Possible causes:

- Application crashed
 - Server powered off
 - Process not running
-

Example 3

Memory:

98%

Likely causes:

- Memory leak
 - Java heap exhaustion
 - Too many worker processes
-

Example 4

Disk Usage:

100%

Very common in production.

When the root filesystem is full:

- Application cannot write logs.
 - Databases fail.
 - Services stop.
 - SSH may become unstable.
-

Step 4 – Connect to the EC2 Instance

In modern production environments, I prefer **AWS Systems Manager Session Manager** instead of SSH because it doesn't require opening port 22 and provides better auditing.

If SSH is used:

```
ssh ec2-user@<public-ip>
```

Step 5 – Check System Resources

First command:

```
top
```

or

```
htop
```

Check:

- CPU
- Memory
- Load Average

Example:

```
Load Average
```

```
45
```

On a 4-vCPU server, a load average of 45 indicates severe resource contention.

Memory

```
free -h
```

Example:

Available Memory

50 MB

Likely issue:

- Out Of Memory (OOM)

The Linux OOM Killer may terminate the application.

Disk

`df -h`

Example:

Filesystem

100%

Very common production issue.

Step 6 – Check Running Services

Suppose the application uses Nginx.

`systemctl status nginx`

Possible output:

inactive (dead)

Restart:

`sudo systemctl restart nginx`

If it fails again:

`journalctl -xe`

Review the logs to identify the root cause.

Step 7 – Review Application Logs

Examples:

Nginx:

```
tail -100 /var/log/nginx/error.log
```

Apache:

```
tail -100 /var/log/httpd/error_log
```

Java:

```
tail -100 catalina.out
```

Node.js:

```
pm2 logs
```

Spring Boot:

```
tail -100 application.log
```

Common findings:

```
Database Connection Timeout
OutOfMemoryError
Address already in use
Permission Denied
SSL Certificate Expired
```

Each error guides the next troubleshooting step.

Step 8 – Verify the Application Port

Check if the application is listening on the expected port.

```
ss -tulnp
```

or

```
netstat -tulnp
```

Example:

Expected:

```
8080
```

But nothing is listening.

The application likely never started.

Step 9 – Verify Database Connectivity

Suppose the application uses an RDS database.

Test connectivity:

```
nc -zv <rds-endpoint> 3306
```

or

```
telnet <rds-endpoint> 3306
```

If it fails:

Possible causes:

- Security Group
 - NACL
 - Route Table
 - Database unavailable
 - Wrong endpoint
-

Step 10 – Verify Security Groups

Check:

ALB Security Group:

Allow

443

↓

EC2 Security Group:

Allow

443 from ALB Security Group

↓

RDS Security Group:

Allow

3306 from EC2 Security Group

Common mistake:

Someone changes the rule to:

3306

from

0.0.0.0/0

or removes the EC2 Security Group reference entirely.

Step 11 – Check Recent Changes

Most production incidents are caused by recent changes.

I review:

- Jenkins pipeline history
- Git commits
- Terraform applies
- CloudFormation stack events
- Auto Scaling activity
- CloudTrail events

If a deployment started just before the outage, I consider a rollback.

Step 12 – Recovery Strategy

Depending on the root cause:

- Restart the application service.
- Replace the unhealthy EC2 instance.
- Roll back the deployment.
- Launch a new instance from the latest AMI.
- Scale out using the Auto Scaling Group.
- Restore from backup if data corruption occurred.

The key is to restore service quickly while continuing root cause analysis.

Production Best Practices

To reduce the impact of such incidents:

- Run multiple EC2 instances across at least two Availability Zones.
 - Use an Auto Scaling Group.
 - Enable ALB health checks.
 - Use CloudWatch alarms for CPU, memory, disk, and status checks.
 - Prefer Systems Manager Session Manager over SSH.
 - Automate deployments with rollback capabilities.
 - Keep infrastructure in Terraform or CloudFormation.
 - Store logs centrally (CloudWatch Logs, Splunk, or ELK).
-

Common Follow-up Questions

Q1: What if the EC2 instance is healthy but the ALB marks it unhealthy?

Possible causes:

- Wrong health check path.
 - Application not listening on the expected port.
 - Security Group blocking ALB traffic.
 - Application returning HTTP 500.
 - Health check timeout too low.
-

Q2: What if CPU utilization is only 10%, but the website is still slow?

Possible causes:

- Database bottleneck.
 - High disk I/O.
 - Network latency.
 - Thread pool exhaustion.
 - External API delays.
 - Application deadlock.
-

Q3: What if the instance cannot be reached via SSH?

Troubleshoot in this order:

1. EC2 status checks.
2. Security Group (port 22).

3. Network ACL.
 4. Route Table.
 5. Internet Gateway or NAT (depending on subnet).
 6. Key pair.
 7. Systems Manager Session Manager (if configured).
 8. EC2 Serial Console (supported instances).
 9. Rescue using the root EBS volume on another instance if necessary.
-

Interview-Ready Answer (2–3 Minutes)

"When I receive a production outage alert, I first determine the scope of the issue—whether it's affecting all users or only a subset, and whether there were any recent deployments or infrastructure changes. Since the application is behind an ALB, I start by checking the ALB target health. If the targets are unhealthy, I move to the EC2 layer and verify the instance status checks and CloudWatch metrics such as CPU, memory, disk usage, and network activity. Next, I connect to the instance using AWS Systems Manager Session Manager or SSH, inspect system resources with commands like `top`, `free -h`, and `df -h`, verify that the application service is running, and review application logs for errors. I then confirm the application is listening on the correct port and verify connectivity to dependencies such as the database. I also review Security Groups, Network ACLs, and any recent deployments or Terraform changes. Based on the findings, I restore service by restarting the application, replacing the instance, scaling out, or rolling back the deployment. Finally, I document the root cause and implement preventive measures such as CloudWatch alarms, Auto Scaling, and automated rollback."

Scenario 2 – Production Application is Slow During Peak Traffic

Difficulty: ★ ★ ★ ★ ★

Experience Level: 5–8 Years DevOps Engineer

AWS Services Covered:

- EC2
- Auto Scaling Group (ASG)
- Application Load Balancer (ALB)
- CloudWatch
- RDS
- VPC
- Security Groups
- IAM
- CloudTrail

Interview Question

Your production application is running on EC2 behind an Application Load Balancer. During normal hours the application works perfectly. Every day between 7 PM and 10 PM, users complain that the application becomes very slow. Some pages take 20–30 seconds to load and users occasionally receive 504 Gateway Timeout errors. Walk me through your complete troubleshooting approach.

What the Interviewer is Testing

The interviewer wants to know:

- Can you troubleshoot systematically?
- Do you understand AWS architecture?
- Can you differentiate between infrastructure and application problems?
- Can you identify the bottleneck instead of guessing?
- Can you think like a Senior DevOps Engineer during a production incident?

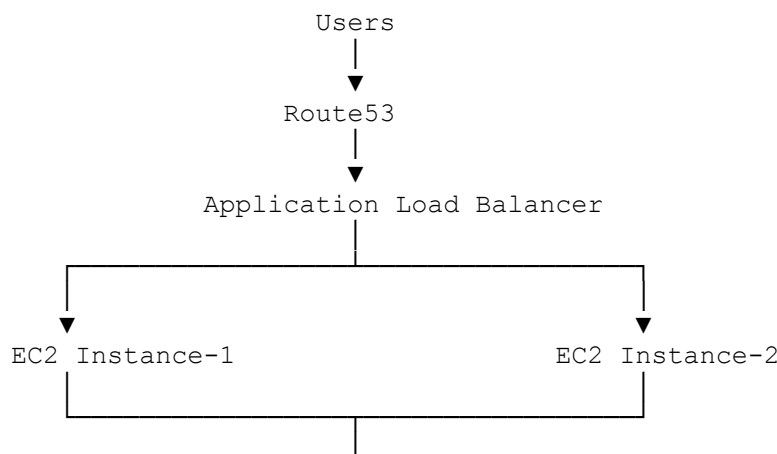
A wrong answer would be:

"I'll restart the EC2."

A senior engineer first identifies the root cause and then decides on the corrective action.

Understanding the Architecture

Let's assume the application architecture looks like this.



▼
Amazon RDS

During normal hours:

500 Users

During evening:

15000 Users

Problem:

Application becomes slow.

My Troubleshooting Strategy

I always troubleshoot from the top layer to the bottom layer.

User

↓

DNS

↓

ALB

↓

EC2

↓

Application

↓

Database

↓

External APIs

Never assume EC2 is the issue.

Step 1 – Gather Information

Before opening AWS Console, I ask a few questions.

- Is the application slow for all users?
- Does the issue happen every day?
- Is only one API slow?
- Was there a deployment today?
- Any Terraform changes?
- Any Auto Scaling events?
- Any CloudWatch alarms?
- Is the issue affecting one Availability Zone or both?

Suppose the application slows down only every evening.

That tells me this is most likely:

- Traffic-related
- Capacity-related
- Performance-related

Step 2 – Check CloudWatch Metrics

Open

CloudWatch

↓

Metrics

↓

EC2

Check the following metrics:

- CPU Utilization
- Network In
- Network Out
- Disk Read Operations
- Disk Write Operations
- Status Check Failed
- Memory Utilization (CloudWatch Agent)
- Disk Usage (CloudWatch Agent)

Case 1 – CPU Utilization

Morning

15%

Evening

98%

Meaning:

The EC2 instance is overloaded.

Possible reasons:

- High traffic
- Infinite loop
- Bad application code
- Too many requests
- Background jobs

AWS CLI

```
aws cloudwatch get-metric-statistics \
--namespace AWS/EC2 \
--metric-name CPUUtilization \
--dimensions Name=InstanceId,Value=i-123456 \
--statistics Average \
--period 300 \
--start-time 2026-07-01T10:00:00Z \
--end-time 2026-07-01T14:00:00Z
```

Step 3 – Check Auto Scaling Group

Go to

EC2

↓

Auto Scaling Groups

Check

Minimum Capacity

Desired Capacity

Maximum Capacity

Current Capacity

Suppose

Minimum = 2

Desired = 2

Maximum = 2

Current = 2

CPU reaches

98%

But ASG doesn't launch new instances.

Why?

Because the Maximum Capacity is already reached.

Solution

Increase

Maximum Capacity

2

↓

10

Configure Target Tracking Policy

CPU > 70%

↓

Launch 1 Instance

Interview Follow-up

Interviewer:

Why don't you simply increase the EC2 size?

Answer

Vertical scaling has limitations and often requires downtime.

Horizontal scaling using Auto Scaling Groups provides:

- High Availability
 - Elasticity
 - Automatic recovery
 - Better fault tolerance
-

Step 4 – Check ALB Metrics

Open

EC2

↓

Load Balancers

↓

Monitoring

Review:

- Request Count
 - Target Response Time
 - HTTP 5XX
 - HTTP 4XX
 - Healthy Hosts
 - Unhealthy Hosts
-

Suppose

Morning

200 Requests/sec

Evening

6000 Requests/sec

Target Response Time

Morning

150 ms

Evening

18 Seconds

Meaning

Backend servers are taking too long to respond.

Step 5 – Check Target Group

Go to

Target Groups

↓

Targets

Check

Healthy

or

Unhealthy

If targets are unhealthy,

ALB will return

502

or

504

Possible causes

- Application crash
 - Health check failure
 - Wrong port
 - Wrong health check path
-

Step 6 – Login to EC2

Use

AWS Systems Manager Session Manager

or SSH.

First command

```
top
```

Output

Load Average

22.5

Server

4 vCPU

A load average of

22

means the server is overloaded.

Step 7 – Check Memory

```
free -h
```

Example

Total

8 GB

Available

150 MB

Meaning

Memory is exhausted.

Linux may start swapping.

Swapping causes major performance degradation.

Step 8 – Check Disk Usage

```
df -h
```

Output

```
Filesystem
```

```
98%
```

A nearly full disk affects:

- Logging
- Temporary files
- JVM
- Docker
- Package updates

Performance degrades significantly.

Step 9 – Check Running Processes

```
top -o %CPU
```

or

```
ps -eo pid,ppid,cmd,%cpu,%mem --sort=-%cpu
```

Output

```
java
```

```
95%
```

Now investigate why Java is consuming CPU.

Could be

- Infinite loop
 - Garbage Collection
 - Memory leak
 - High traffic
-

Step 10 – Check Application Logs

Java

```
tail -100 application.log
```

Nginx

```
tail -100 /var/log/nginx/error.log
```

Apache

```
tail -100 /var/log/httpd/error_log
```

Possible findings

```
Database Timeout  
Connection Refused  
Thread Pool Exhausted  
OutOfMemoryError
```

Logs provide the actual root cause.

Step 11 – Check Database (RDS)

Open

Amazon RDS

↓

Monitoring

Review:

- CPU
- Memory
- Connections
- Read IOPS
- Write IOPS
- Storage
- Disk Queue Depth

Suppose

CPU

99%

Meaning

Database is the bottleneck.

Scaling EC2 won't solve the issue.

Step 12 – Check Slow Queries

Enable Slow Query Log.

Example

```
SELECT *  
FROM Orders  
WHERE CustomerName LIKE '%John%';
```

Without an index,

MySQL scans the entire table.

Solution

```
CREATE INDEX idx_customer_name  
ON Orders (CustomerName);
```

Performance improves dramatically.

Step 13 – Check Connection Pool

Bad design

10000 Users

↓

10000 Database Connections

Good design

10000 Users

↓

100 Reusable Connections

Use connection pooling.

Examples

- HikariCP

- PgBouncer
 - c3p0
-

Step 14 – Check External Dependencies

Suppose

Application

↓

Payment Gateway

↓

Response Time

20 Seconds

The application appears slow,

but the external API is causing delays.

Always verify downstream dependencies.

Step 15 – Check Recent Changes

Most production incidents occur after changes.

Review

- Jenkins Deployment
- Git Commits
- Terraform Apply
- CloudTrail
- Auto Scaling Activity

Questions

- Was a new release deployed?
 - Was instance type changed?
 - Was Auto Scaling modified?
-

Immediate Recovery Plan

If customers are impacted,

I focus on restoring service first.

Actions

- Increase Auto Scaling capacity.
 - Add more EC2 instances.
 - Restart failed application services.
 - Scale RDS if required.
 - Roll back recent deployment.
 - Enable caching.
-

Long-Term Solution

- Configure Target Tracking Auto Scaling.
 - Use Amazon ElastiCache (Redis).
 - Optimize SQL queries.
 - Add Read Replicas.
 - Enable CloudWatch Alarms.
 - Perform Load Testing.
 - Enable Application Performance Monitoring.
-

Real Production Incident

One production application slowed down every evening.

Investigation showed

EC2 CPU

20%

Memory

Normal

ALB

Healthy

RDS CPU

99%

Root Cause

A reporting job executed every evening.

It performed a full table scan on a table with 300 million rows.

Solution

- Added indexes.
- Moved reporting job to midnight.
- Created a Read Replica.

No EC2 scaling was required.

Common Mistakes

- ✗ Restart EC2 immediately.
 - ✗ Scale EC2 without checking RDS.
 - ✗ Ignore CloudWatch.
 - ✗ Assume CPU is always the bottleneck.
 - ✗ Ignore application logs.
 - ✗ Ignore recent deployments.
-

Production Best Practices

- Multi-AZ Architecture
- Auto Scaling Group
- Target Tracking Scaling Policy
- CloudWatch Dashboards
- CloudWatch Alarms
- ElastiCache
- Read Replicas
- Database Connection Pooling
- Blue/Green Deployments
- Regular Load Testing

Common Interview Follow-up Questions

Q1. CPU is only 15% but users still complain.

Possible causes

- Database bottleneck
- Slow SQL
- External API delay
- DNS issue
- Thread exhaustion
- Network latency

Q2. Auto Scaling didn't launch new instances.

Possible reasons

- Maximum Capacity reached
- Scaling Policy incorrect
- CloudWatch Alarm missing
- Launch Template issue
- Health Check failure

Q3. Why did ALB return 504 Gateway Timeout?

Possible reasons

- Backend took too long
- Application hung
- Database timeout
- Health check issue
- Network issue

Senior DevOps Interview Answer (2–3 Minutes)

"When an application becomes slow only during peak traffic, I first determine whether the issue is due to increased load or a recent infrastructure or application change. I begin by reviewing CloudWatch metrics for EC2, ALB, and RDS to check CPU, memory, disk, request count, response time, and database performance. Next, I verify that the Auto Scaling

Group is scaling correctly and that it hasn't reached its maximum capacity. I then log into the EC2 instance using Systems Manager or SSH to inspect CPU, memory, disk usage, running processes, and application logs. If the compute layer is healthy, I investigate the database by reviewing CPU, connections, IOPS, and slow queries. I also validate any external API dependencies and recent deployments. Based on the findings, I may increase Auto Scaling capacity, optimize SQL queries, introduce ElastiCache, scale the RDS instance, or roll back a problematic release. Finally, I implement preventive measures such as CloudWatch alarms, load testing, connection pooling, and performance monitoring."

Scenario 3 – Unable to SSH into EC2 Instance

Difficulty: ☆ ☆ ☆ ☆ ☆

Services Covered: EC2, Security Groups, NACL, Route Tables, Internet Gateway, Systems Manager, VPC

Interview Question

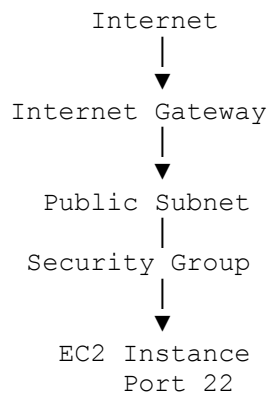
Your production EC2 instance is running, but you're unable to SSH into it. How will you troubleshoot?

What the Interviewer is Testing

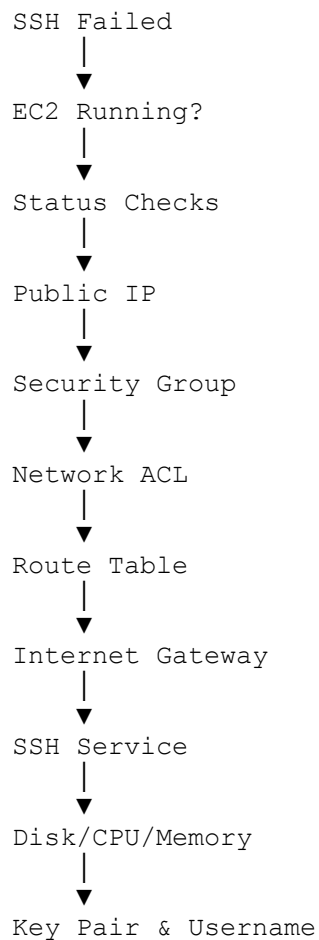
The interviewer wants to check your understanding of:

- EC2
 - VPC Networking
 - Security Groups
 - NACL
 - Route Tables
 - Internet Gateway
 - SSH Service
 - Key Pair
 - AWS Systems Manager
 - Linux Troubleshooting
-

Architecture



Troubleshooting Flow



Step 1 – Check EC2 State

Go to:

EC2
→ Instances

Verify:

- Instance State = **Running**
- Not Stopped
- Not Terminated

CLI

```
aws ec2 describe-instances --instance-ids i-xxxxxxx
```

Step 2 – Check Status Checks

Verify both status checks.

[System Status Check](#)

Checks AWS infrastructure:

- Hardware
- Hypervisor
- Network

[Instance Status Check](#)

Checks Operating System:

- Boot process
- Kernel
- Filesystem

If either fails, SSH may not work.

Step 3 – Verify Public IP

Check:

- Has the Public IP changed?
- Is an Elastic IP attached?

Many times the instance is stopped and started, resulting in a new Public IP.

Step 4 – Check Security Group

Inbound Rule must allow:

```
TCP
Port 22
Source: Your IP or Bastion Host
```

Example

```
22
203.xxx.xxx.xxx/32
```

Wrong configuration:

```
Port 22 Missing
```

OR

```
Wrong Source CIDR
```

CLI

```
aws ec2 describe-security-groups
```

Step 5 – Check Network ACL

Ensure:

Inbound

```
22 → ALLOW
```

Outbound

```
1024-65535 → ALLOW
```

SSH uses ephemeral ports for responses.

Step 6 – Check Route Table

Public subnet must have:

```
0.0.0.0/0
```

↓

Internet Gateway

Without this route, SSH won't work.

Step 7 – Check Internet Gateway

Verify:

- IGW is attached to the VPC.
- Public subnet uses that VPC.

Without an Internet Gateway, a Public IP is useless.

Step 8 – Check SSH Service

If Systems Manager is available:

```
systemctl status sshd
```

If stopped:

```
sudo systemctl restart sshd
```

Enable on boot:

```
sudo systemctl enable sshd
```

Step 9 – Check System Resources

CPU

```
top
```

Memory

```
free -h
```

Disk

```
df -h
```

If:

- CPU = 100%
- Memory exhausted
- Disk = 100%

SSH may become unresponsive.

Step 10 – Check SSH Logs

Amazon Linux

```
journalctl -u sshd
```

Ubuntu

```
cat /var/log/auth.log
```

Common errors:

- Permission denied
- Authentication failed
- Too many connections
- Invalid user

Step 11 – Verify Key Pair & Username

Correct username:

OS	Username
Amazon Linux	ec2-user
Ubuntu	ubuntu
RHEL	ec2-user
Debian	admin

Wrong key pair or username results in:

```
Permission denied (publickey)
```

If SSH Still Doesn't Work

Use:

- AWS Systems Manager Session Manager
- EC2 Serial Console (if enabled)
- Detach the EBS volume, attach it to another EC2 instance, fix the issue, and reattach it.

AWS CLI Commands

Describe Instance

```
aws ec2 describe-instances
```

Describe Security Group

```
aws ec2 describe-security-groups
```

Describe Route Tables

```
aws ec2 describe-route-tables
```

Linux Commands

```
top
free -h
df -h
systemctl status sshd
journalctl -u sshd
ss -tulnp
```

Common Root Causes

Root Cause	Resolution
Port 22 blocked	Update Security Group
Wrong Public IP	Use current IP/Elastic IP
SSH service stopped	Restart sshd
Disk full	Clean logs / Extend EBS
CPU 100%	Kill process / Scale
Wrong key pair	Use correct private key

Root Cause	Resolution
Wrong username	Use correct OS username
Route missing	Update Route Table

Production Best Practices

- Use **AWS Systems Manager Session Manager** instead of exposing SSH.
 - Restrict port 22 to office/VPN IPs.
 - Use a Bastion Host if SSH is required.
 - Never allow `0.0.0.0/0` for SSH in production.
 - Enable CloudWatch alarms for CPU, memory, and disk usage.
-

Common Follow-up Questions

Q1. Website is working but SSH is not. Why?

Possible causes:

- Port 22 blocked
 - SSH service stopped
 - Wrong key pair
 - Wrong username
 - Security Group issue
-

Q2. Website and SSH both are down.

Possible causes:

- EC2 stopped
 - Kernel panic
 - Status check failure
 - Disk full
 - Application crash
-

Q3. How do you access an EC2 instance if SSH is blocked?

Answer:

- Systems Manager Session Manager
- EC2 Serial Console

- Detach and repair the EBS volume
-

2-Minute Interview Answer

"If I'm unable to SSH into an EC2 instance, I first verify that the instance is running and both the System and Instance Status Checks are healthy. Then I check whether the Public IP or Elastic IP has changed. Next, I verify the Security Group allows inbound traffic on port 22 from my IP, followed by checking the Network ACL, Route Table, and Internet Gateway. If networking is correct, I use AWS Systems Manager Session Manager to access the instance and check whether the SSH service is running. I also verify CPU, memory, and disk usage because resource exhaustion can make SSH unresponsive. Finally, I review SSH logs, confirm I'm using the correct key pair and username, identify the root cause, restore access, and document preventive actions."

Scenario 4 – EC2 CPU Utilization is 100%

Difficulty: ★ ★ ★ ★ ★

Services Covered: EC2, CloudWatch, Auto Scaling, ALB, Systems Manager, CloudTrail

Interview Question

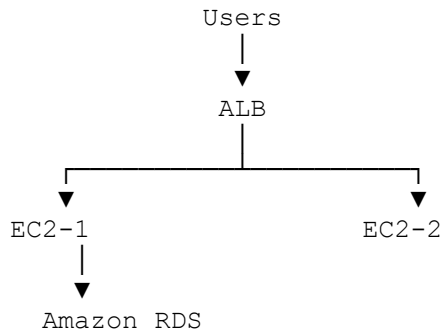
Your production EC2 instance is running, but CloudWatch shows CPU utilization at 100%. Users report that the application is very slow. How will you troubleshoot and resolve the issue?

What the Interviewer is Testing

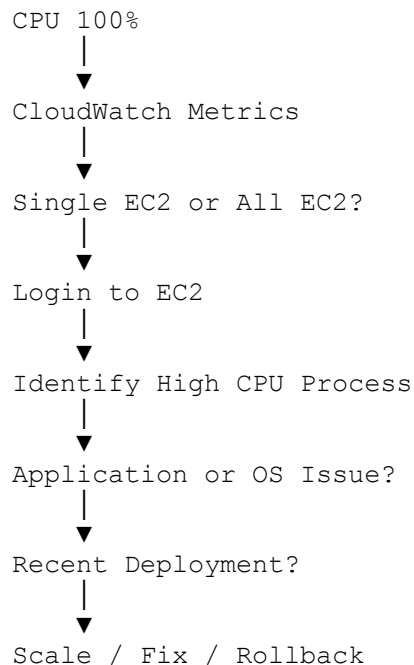
The interviewer wants to evaluate your understanding of:

- CloudWatch Monitoring
 - Linux Performance Troubleshooting
 - EC2 Resource Utilization
 - Auto Scaling
 - Application Performance
 - Root Cause Analysis
-

Production Architecture



Troubleshooting Flow



Step 1 – Verify CloudWatch Metrics

Open:

CloudWatch
→ Metrics
→ EC2

Check:

- CPU Utilization
- Network In/Out

- Disk Read/Write
- Status Checks

Questions to ask:

- Is only one EC2 affected?
- Are all EC2 instances showing high CPU?
- Did CPU suddenly spike after deployment?
- Is this happening only during peak traffic?

Step 2 – Check Auto Scaling Group

Verify:

- Desired Capacity
- Current Capacity
- Maximum Capacity

Example:

`Desired = 2`

`Current = 2`

`Maximum = 2`

If CPU is continuously above 80% and no new instance is launched, Auto Scaling may be misconfigured or the maximum capacity has already been reached.

Step 3 – Login to EC2

Preferred:

- AWS Systems Manager Session Manager

or

`ssh ec2-user@<ip>`

Step 4 – Identify High CPU Process

Check CPU usage:

`top`

or

`htop`

or

`ps -eo pid,ppid,cmd,%cpu --sort=-%cpu`

Example:

```
java          96%
```

This indicates that the Java application is consuming most of the CPU.

Step 5 – Analyze the Cause

Common reasons for high CPU:

High User Traffic

Example:

Morning

500 Users

↓

Evening

15000 Users

Solution:

- Auto Scaling
 - Load Balancing
-

Infinite Loop in Application

Symptoms:

- One process always at 100%
- CPU remains high even with low traffic

Solution:

- Identify buggy code
- Restart application
- Deploy fixed version

Background Job

Examples:

- Backup
- Log processing
- Report generation

Solution:

- Schedule during off-peak hours
- Optimize the job

DDoS or Bot Traffic

Check:

- ALB Request Count
- WAF Logs
- CloudFront Logs

Solution:

- Enable AWS WAF
- Configure rate limiting

Malware / Crypto Mining

Symptoms:

Unknown process consuming CPU.

Check:

```
top  
ps aux
```

Investigate and isolate the instance if malicious activity is suspected.

Step 6 – Check Application Logs

Examples:

Java

```
tail -100 application.log
```

Nginx

```
tail -100 /var/log/nginx/error.log
```

Look for:

- Infinite retries
 - Thread exhaustion
 - Database timeout
 - High request volume
-

Step 7 – Check Database

Sometimes the CPU is high because the application is waiting on slow database queries.

Verify:

- RDS CPU
- Active Connections
- Slow Query Log

If the database is overloaded, optimize queries or scale the database instead of only scaling EC2.

Step 8 – Check Recent Changes

Review:

- Jenkins deployments
- Git commits
- Terraform apply
- CloudTrail events

A recent deployment may have introduced inefficient code.

Immediate Recovery

Depending on the cause:

Problem	Resolution
High traffic	Scale Auto Scaling Group
Application bug	Restart/Rollback
Background job	Stop or reschedule
DDoS	Enable WAF
Malware	Isolate instance
Database bottleneck	Optimize or scale RDS

AWS CLI Commands

CloudWatch Metrics

```
aws cloudwatch get-metric-statistics
```

Describe EC2

```
aws ec2 describe-instances
```

Auto Scaling

```
aws autoscaling describe-auto-scaling-groups
```

Linux Commands

CPU

```
top
```

Process

```
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu
```

Load Average

uptime

Memory

free -h

Disk

df -h

Common Root Causes

Root Cause	Resolution
High traffic	Auto Scaling
Infinite loop	Fix application code
Heavy background jobs	Schedule off-peak
Database bottleneck	Optimize queries
DDoS attack	AWS WAF
Malware	Remove and replace instance

Production Best Practices

- Configure Target Tracking Auto Scaling.
 - Set CloudWatch alarms for CPU > 70%.
 - Perform load testing before production releases.
 - Enable AWS WAF.
 - Monitor application performance with tools like Dynatrace or Prometheus.
 - Keep application code optimized.
-

Common Follow-up Questions

Q1. CPU is 100%, but Memory is only 30%. What does it indicate?

Likely causes:

- CPU-intensive application
- Infinite loop

- Heavy computations
- Background jobs

Q2. CPU is only 20%, but the application is slow.

Possible causes:

- Database bottleneck
- Network latency
- External API delay
- Disk I/O issue

Q3. How do you prevent high CPU in production?

- Auto Scaling
- Load testing
- CloudWatch alarms
- Performance tuning
- Optimized SQL queries
- Caching using Redis/ElastiCache

2-Minute Interview Answer

"If CPU utilization reaches 100% on a production EC2 instance, I first verify the CloudWatch metrics to determine whether the issue affects a single instance or the entire Auto Scaling Group. I then check if Auto Scaling is functioning correctly and whether it has reached its maximum capacity. Next, I log into the instance using Systems Manager or SSH and identify the process consuming CPU using commands like `top` or `ps`. Based on the findings, I determine whether the issue is caused by high traffic, an application bug, a background job, a database bottleneck, or malicious activity. I review application logs, recent deployments, and database performance before deciding on the resolution, such as scaling the infrastructure, rolling back a deployment, optimizing queries, or fixing the application. Finally, I implement preventive measures such as CloudWatch alarms, Auto Scaling, load testing, and application performance monitoring."

Scenario 5 – EC2 Disk Full Causing Production Outage

Difficulty: ☆ ☆ ☆ ☆ ☆

Services Covered: EC2, EBS, CloudWatch, Systems Manager, Auto Scaling

Interview Question

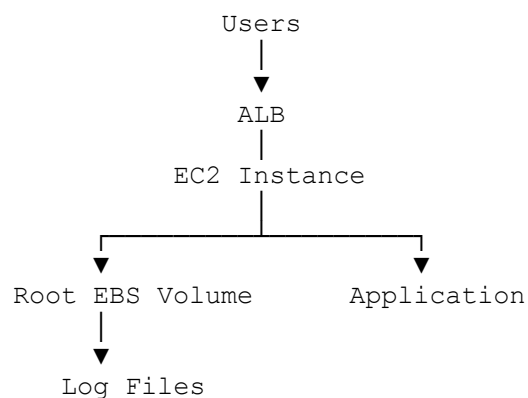
Your production application suddenly goes down. You log in to the EC2 instance and find that the root volume is 100% utilized. How will you troubleshoot and resolve the issue?

What the Interviewer is Testing

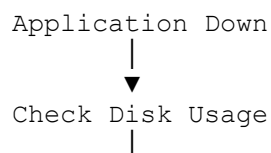
The interviewer wants to evaluate your understanding of:

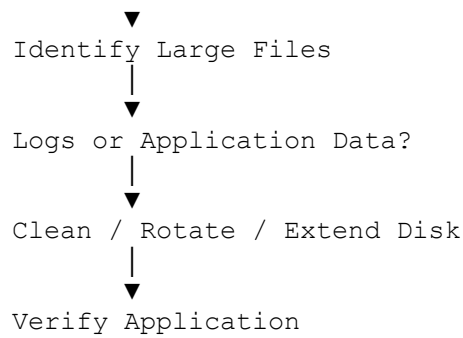
- Linux Disk Management
 - EBS Volumes
 - CloudWatch Monitoring
 - Log Management
 - Production Troubleshooting
 - Capacity Planning
-

Production Architecture



Troubleshooting Flow





Step 1 – Verify Disk Usage

Login to EC2 using Systems Manager or SSH.

Check disk usage.

```
df -h
```

Example

Filesystem	Size	Used	Avail	Use%
/dev/xvda1	20G	20G	0G	100%

This confirms the disk is completely full.

Step 2 – Identify Which Directory is Full

Run:

```
du -sh /*
```

or

```
du -sh /var/*
```

Example

```
/var/log
```

```
18 GB
```

Now you know logs are consuming disk space.

Step 3 – Find Large Files

```
find / -type f -size +500M
```

Example

```
application.log
```

```
8 GB
```

```
messages
```

```
5 GB
```

These files are the primary candidates for cleanup.

Step 4 – Check Log Rotation

Check whether log rotation is configured.

```
cat /etc/logrotate.conf
```

Check application-specific configuration:

```
ls /etc/logrotate.d/
```

If log rotation is missing or misconfigured, log files continue growing until the disk is full.

Step 5 – Clean Up Disk

If logs are no longer required:

```
truncate -s 0 /var/log/application.log
```

or

```
rm old.log
```

Do not delete active log files blindly. Truncating is often safer because the application can continue writing to the same file handle.

Step 6 – Check Deleted but Open Files

Sometimes a file is deleted but the application still holds it open.

Check:

```
lsof | grep deleted
```

If found, restart the application so Linux releases the disk space.

Step 7 – Check Inode Usage

Sometimes space is available, but inodes are exhausted.

```
df -i
```

If inode usage is 100%, investigate directories with millions of small files.

Step 8 – Extend the EBS Volume

If the disk is genuinely too small:

[AWS Console](#)

EC2

↓

Elastic Block Store

↓

Volumes

↓

Modify Volume

Increase

20 GB

↓

50 GB

Then extend the filesystem.

For XFS:


```
sudo xfs_growfs -d /
```

For EXT4:

```
sudo resize2fs /dev/xvda1
```

Step 9 – Verify Application

Check whether the application is running.

```
systemctl status nginx
```

or

```
systemctl status httpd
```

or

```
systemctl status docker
```

If stopped:

```
sudo systemctl restart nginx
```

Verify application logs again to ensure no further disk-related errors occur.

Step 10 – Verify CloudWatch

Review:

- Disk Usage
- Disk Read/Write
- CPU
- Memory

Confirm that the application has recovered and no additional alerts are firing.

AWS CLI Commands

Describe Volumes

```
aws ec2 describe-volumes
```

Modify Volume

```
aws ec2 modify-volume \  
--volume-id vol-xxxxxxxx \  
--size 50
```

Linux Commands

Check Disk

```
df -h
```

Directory Sizes

```
du -sh /*
```

Find Large Files

```
find / -type f -size +500M
```

Check Inodes

```
df -i
```

Deleted Open Files

```
lsof | grep deleted
```

Common Root Causes

Root Cause	Resolution
Application logs	Configure log rotation
Debug logging enabled	Disable debug logs
Backup files	Remove old backups
Docker images/containers	Clean unused images and containers
Large temporary files	Remove <code>/tmp</code> files
Small EBS volume	Extend EBS volume

Production Best Practices

- Enable CloudWatch alarms for disk usage (>80%).
 - Configure `logrotate`.
 - Store application logs in CloudWatch Logs or a centralized logging system.
 - Avoid writing excessive logs at DEBUG level in production.
 - Monitor EBS utilization regularly.
 - Implement automatic cleanup for temporary files.
 - Use larger EBS volumes with room for growth.
-

Common Follow-up Questions

Q1. How do you know which folder is consuming the most space?

```
du -sh /*
```

Q2. Why use `truncate` instead of `rm`?

Because deleting an active log file may not immediately free space if the process still has the file open. `truncate` clears the file contents while keeping the file handle intact.

Q3. What if you increased the EBS volume but `df -h` still shows the old size?

Because increasing the EBS volume only expands the block device. You must also extend the filesystem using:

- `xfs_growfs` (XFS)
 - `resize2fs` (EXT4)
-

Q4. What if `df -h` shows free space but the application still reports "No space left on device"?

Check inode usage:

```
df -i
```

The issue may be inode exhaustion rather than disk capacity.

2-Minute Interview Answer

"If a production EC2 instance becomes unavailable because the disk is full, I first verify disk utilization using `df -h`. Then I identify which directories consume the most space using `du -sh` and locate large files with `find`. I check whether log files, backups, Docker artifacts, or

temporary files are responsible. I also verify whether log rotation is configured and check for deleted but open files using `ls -oF`. If the disk size itself is insufficient, I extend the EBS volume and then grow the filesystem using the appropriate Linux command. Finally, I restart affected services if required, verify application health, and implement preventive measures such as CloudWatch alarms, log rotation, and centralized logging."

Scenario 6: EC2 Instance Unhealthy Behind an ALB

Interview Question

An EC2 instance behind an Application Load Balancer (ALB) becomes unhealthy, and the ALB stops sending traffic to it. How would you troubleshoot it?

Answer

When an EC2 instance becomes unhealthy behind an ALB, I follow a structured troubleshooting approach to identify whether the issue is related to the application, networking, health checks, or the EC2 instance itself.

Step 1: Verify Target Health

First, I check the target group's health status in the AWS Console or using the AWS CLI.

```
aws elbv2 describe-target-health --target-group-arn <target-group-arn>
```

This tells me why the instance is marked unhealthy (e.g., health check failed, timeout, or connection error).

Step 2: Validate Health Check Configuration

I verify:

- Health check path (e.g., `/health`)
- Port
- Protocol (HTTP/HTTPS)
- Timeout and interval
- Success response code (usually 200)

A wrong path or port is one of the most common reasons for health check failures.

Step 3: Check if the Application is Running

SSH into the EC2 instance and verify the application service.

```
systemctl status nginx
```

or

```
systemctl status myapp
```

If the service is stopped, restart it.

```
sudo systemctl restart myapp
```

Step 4: Test the Application Locally

Verify that the application is responding correctly.

```
curl localhost:8080/health
```

If the endpoint doesn't return **HTTP 200**, the issue is with the application.

Step 5: Verify Network Configuration

Check that:

- EC2 Security Group allows traffic from the ALB Security Group.
 - Network ACLs are not blocking traffic.
 - Target Group port matches the application's listening port.
-

Step 6: Review Logs and System Resources

Check application and system logs for errors.

Useful commands:

```
journalctl -xe  
df -h  
free -m  
top
```

Look for:

- Application crashes
- High CPU or memory usage
- Full disk

- Database connection failures

Step 7: Verify Dependencies

If the application depends on services such as a database, Redis, or external APIs, ensure those services are reachable.

Example:

```
nc -zv database-server 3306
```

Step 8: Confirm Recovery

After fixing the issue:

- Restart the application if required.
- Verify the health endpoint.
- Wait for the ALB to complete successful health checks.
- Confirm the instance status changes to **Healthy**.

Real-Time Production Scenario

In one production deployment, an EC2 instance became unhealthy after a new application release. Investigation showed that the application was listening on **port 8080**, while the Target Group was configured to check **port 80**. After updating the Target Group to use port **8080**, the health checks passed, and the ALB automatically started routing traffic back to the instance.

Best Practices

- Implement a dedicated `/health` endpoint.
- Configure correct health check path and port.
- Allow only the ALB Security Group to access the application.
- Monitor CloudWatch metrics such as **HealthyHostCount** and **TargetResponseTime**.
- Use Auto Scaling to replace unhealthy instances automatically.

Interview Summary

"When an EC2 instance becomes unhealthy behind an ALB, I first check the Target Group health status and reason for failure. Then I validate the health check configuration, verify the application is running, test the health endpoint locally, and check Security Groups, Network

ACLs, logs, system resources, and dependencies. After resolving the issue, I verify that the health checks pass and the ALB resumes routing traffic to the instance."

Scenario 7: Unable to SSH into an EC2 Instance

Interview Question

Your production EC2 instance is unreachable, and SSH connection times out. How would you troubleshoot it?

Answer

If I'm unable to SSH into an EC2 instance, I troubleshoot the issue step by step to identify whether it's related to networking, security, or the instance itself.

Step 1: Check EC2 Status

Verify the instance is **Running** and both **System Status** and **Instance Status Checks** are passing.

Step 2: Check Security Group

Ensure the Security Group allows:

- TCP Port **22**
 - Source: My Public IP (or trusted CIDR)
-

Step 3: Check Network Configuration

Verify:

- Public IP or Elastic IP is attached.
 - Route Table has a route to the **Internet Gateway**.
 - Network ACL allows SSH traffic.
-

Step 4: Verify SSH Service

If accessible through **AWS Systems Manager (SSM)** or **EC2 Serial Console**, check that the SSH service is running.

```
sudo systemctl status sshd
```

Restart if necessary.

```
sudo systemctl restart sshd
```

Step 5: Check System Resources

Verify the instance isn't overloaded.

```
df -h
free -m
top
```

Look for:

- Full disk
 - High CPU
 - Low memory
-

Step 6: Verify SSH Credentials

Ensure you're using:

- The correct **.pem key**
 - The correct username (`ec2-user`, `ubuntu`, etc.)
-

Step 7: Recovery

If the issue persists:

- Review system logs.
 - Use **SSM Session Manager** if available.
 - Restart the instance or recover the root EBS volume if required.
-

Real-Time Scenario

A production server became inaccessible after a Security Group update. Investigation showed that **port 22 had been removed**. After restoring the SSH rule, access was immediately restored.

Interview Summary

"I first check the EC2 status and health checks, then verify the Security Group, Network ACL, route table, and public IP. Next, I ensure the SSH service is running, check system resources, and confirm the correct SSH key and username are being used. If needed, I use AWS Systems Manager or recover the instance using EBS."

Scenario 8: EC2 Instance is Running but Performance is Very Slow

Interview Question

Your production EC2 instance is running, but users report that the application is very slow. How would you troubleshoot it?

Answer

If an EC2 instance becomes slow, I first determine whether the issue is related to CPU, memory, disk, network, or the application.

Step 1: Check Resource Utilization

Check CPU, memory, and load on the instance.

`top`

or

`htop`

Look for:

- High CPU
- High memory usage
- High load average

Step 2: Check Disk Space

`df -h`

A full disk can slow down the application and prevent log writing.

Step 3: Check Memory Usage

```
free -m
```

If memory is exhausted, the instance may start swapping, causing poor performance.

Step 4: Check Application Logs

Review application and system logs for errors.

```
journalctl -xe
```

Look for:

- Application crashes
- Database connection issues
- Timeout errors

Step 5: Check CloudWatch Metrics

Monitor:

- CPU Utilization
- Memory (if enabled)
- Disk I/O
- Network In/Out

This helps identify resource bottlenecks.

Step 6: Verify Dependencies

Ensure dependent services such as databases, Redis, or external APIs are healthy and responding normally.

Step 7: Resolve the Issue

Depending on the root cause:

- Restart the application.
- Clean up disk space.
- Stop unnecessary processes.
- Resize the EC2 instance if resources are insufficient.

Real-Time Scenario

A production application became slow because **CPU utilization remained above 95%** due to increased traffic. After analyzing CloudWatch metrics, we upgraded the EC2 instance type and enabled Auto Scaling. Performance returned to normal.

Interview Summary

"When an EC2 instance becomes slow, I first check CPU, memory, disk, and load using Linux commands. Then I review application logs, analyze CloudWatch metrics, and verify dependent services like the database. Based on the root cause, I restart the application, free resources, or scale the instance vertically or horizontally to restore performance."

Scenario 13: EC2 in One AWS Account Needs to Access an S3 Bucket in Another Account

Interview Question

An EC2 instance in Account A needs to access an S3 bucket in Account B. How would you configure it securely?

Answer

The recommended approach is to use **IAM Roles** and an **S3 Bucket Policy** instead of storing AWS access keys on the EC2 instance.

Step 1: Create an IAM Role

Create an IAM role in **Account A** with permissions to access the required S3 bucket.

Attach this role to the EC2 instance.

Step 2: Update the S3 Bucket Policy

In **Account B**, update the bucket policy to allow the IAM role from Account A to access the bucket.

Example permissions:

- `s3:GetObject`
 - `s3:ListBucket`
 - `s3:PutObject` (if upload is required)
-

Step 3: Verify Access

From the EC2 instance, test access using the AWS CLI.

```
aws s3 ls s3://my-bucket
```

If configured correctly, the bucket contents should be listed without using access keys.

Step 4: Troubleshoot if Access Fails

Check:

- IAM Role is attached to the EC2 instance.
 - Bucket Policy allows the IAM role.
 - IAM permissions are correct.
 - Bucket name and Region are correct.
-

Real-Time Scenario

A Jenkins server running on an EC2 instance in **Account A** needed to upload application artifacts to an S3 bucket in **Account B**. We attached an IAM role to the EC2 instance and updated the S3 bucket policy to trust that role. This allowed secure access without storing AWS credentials on the server.

Best Practices

- Use **IAM Roles** instead of access keys.
 - Grant only the minimum required S3 permissions.
 - Use bucket policies for cross-account access.
 - Avoid hardcoding AWS credentials.
-

Interview Summary

"For cross-account S3 access, I attach an IAM role to the EC2 instance in Account A and update the S3 bucket policy in Account B to allow that role. I then verify access

using the AWS CLI. This approach is secure because it avoids storing long-term AWS access keys on the EC2 instance."

✓ **Completed: Scenario 13 of 15**

Next: Scenario 14 – High AWS Bill Due to EC2 Instances, a common production and FinOps interview scenario.

next

Scenario 14: High AWS Bill Due to EC2 Instances

Interview Question

Your AWS bill has increased significantly due to EC2 usage. How would you investigate and reduce the cost?

Answer

If I notice a sudden increase in EC2 costs, I first identify the cause and then optimize resources without affecting the application.

Step 1: Analyze the Cost

Check **AWS Cost Explorer** to identify:

- Which EC2 instances generated the cost.
- When the increase started.
- Which Region and instance type are responsible.

Step 2: Identify Idle or Underutilized Instances

Review **CloudWatch** metrics:

- CPU Utilization
- Network Traffic
- Disk I/O

Instances with consistently low utilization are candidates for downsizing or termination.

Step 3: Check Auto Scaling

Verify that the Auto Scaling Group isn't launching unnecessary instances due to incorrect scaling policies or health check failures.

Step 4: Right-Size Instances

Replace oversized instances with smaller instance types based on actual resource usage.

Step 5: Stop or Terminate Unused Resources

Identify and remove:

- Unused EC2 instances
 - Unattached EBS volumes
 - Unused Elastic IPs
 - Old AMIs and snapshots (if no longer needed)
-

Step 6: Use Cost Optimization Options

- Purchase **Reserved Instances** or **Savings Plans** for predictable workloads.
 - Use **Spot Instances** for non-production or fault-tolerant workloads.
-

Real-Time Scenario

A production AWS bill increased because several **large EC2 instances were running 24/7 with CPU utilization below 10%**. After reviewing CloudWatch metrics, we downsized the instances and purchased a Savings Plan for long-running workloads, reducing EC2 costs significantly.

Best Practices

- Monitor costs using **AWS Cost Explorer** and **AWS Budgets**.
 - Enable CloudWatch monitoring for resource utilization.
 - Schedule non-production instances to stop outside business hours.
 - Regularly review and delete unused resources.
-

Interview Summary

"If EC2 costs increase, I first analyze AWS Cost Explorer to identify the source. Then I review CloudWatch metrics to find underutilized instances, verify Auto Scaling behavior, right-size instances, remove unused resources, and use Reserved Instances, Savings Plans, or Spot Instances where appropriate to optimize costs."

✓ Completed: Scenario 14 of 15

Next: Scenario 15 – Disaster Recovery: Your production EC2 instance or an entire AWS Region fails. How would you recover the application? This is the final and one of the most frequently asked scenario-based interview questions.

next

Scenario 15: Disaster Recovery for EC2 (Instance or Region Failure)

Interview Question

Your production EC2 instance or an entire AWS Region fails. How would you recover the application?

Answer

In a disaster recovery (DR) scenario, my goal is to restore the application with minimal downtime and data loss by following the organization's **RTO (Recovery Time Objective)** and **RPO (Recovery Point Objective)**.

Step 1: Identify the Failure

Determine whether the issue is:

- EC2 instance failure
- Availability Zone failure
- Entire Region failure

Step 2: Recover the EC2 Instance

If only the EC2 instance has failed:

- Launch a new EC2 instance from the latest **AMI**.
- Attach the required **EBS volumes**.

- Verify the application is running.

Step 3: Restore Data

If required:

- Restore data from **EBS snapshots**.
- Restore databases from the latest backup or enable Multi-AZ/replica setup.

Step 4: Update Traffic

Route user traffic to the healthy instance or Region using:

- **Application Load Balancer (ALB)**
- **Route 53 Failover Routing**

Step 5: Verify the Application

Check:

- Application health
- Database connectivity
- Logs
- End-to-end functionality

Ensure users can access the application successfully.

Real-Time Scenario

A production EC2 instance became unavailable due to hardware failure. Since a recent AMI and EBS snapshots were available, a new EC2 instance was launched, the application was restored, and traffic was routed through the Load Balancer. Service was restored within the organization's RTO.

Best Practices

- Take regular **AMI** and **EBS snapshots**.
- Use **Auto Scaling** and **Load Balancers** for high availability.
- Enable **Multi-AZ** for databases.
- Store backups in **S3** and test DR procedures regularly.
- Define and monitor **RTO** and **RPO**.

Interview Summary

"In a disaster recovery scenario, I first identify whether the failure is at the instance, Availability Zone, or Region level. I recover the application by launching a new EC2 instance from an AMI, restoring data from EBS snapshots or backups, and routing traffic using an ALB or Route 53 failover. Finally, I verify the application and ensure recovery meets the defined RTO and RPO."
